

Comprehensive Study Guide

CME 295: Transformers & Large Language Models — Lecture 1

Based on Stanford CME 295 by Afshine & Shervine Amidi

Prepared for: Applied AI/ML Engineers with ML/DL Fundamentals

Contents

1	Topic Map	3
2	Deep-Dive Notes	5
2.1	NLP Task Taxonomy	5
2.1.1	Classification (Text $\rightarrow$ Single Label)	5
2.1.2	Multi-Classification (Text $\rightarrow$ Multiple Labels per Token)	5
2.1.3	Generation (Text $\rightarrow$ Variable-Length Text)	5
2.2	Evaluation Metrics	5
2.2.1	Classification Metrics	5
2.2.2	Generation Metrics	6
2.3	Tokenization	7
2.3.1	Word-Level Tokenization	7
2.3.2	Subword-Level Tokenization	7
2.3.3	Character-Level Tokenization	7
2.4	Token Representation and Word2vec	8
2.4.1	One-Hot Encoding (OHE)	8
2.4.2	Cosine Similarity	8
2.4.3	Word2vec (Mikolov et al., 2013)	8
2.5	Recurrent Neural Networks (RNNs)	9
2.5.1	LSTM (Long Short-Term Memory)	10
2.5.2	The Vanishing Gradient Problem	10
2.6	Attention Mechanism	10
2.7	Self-Attention and the Transformer	11
2.7.1	Self-Attention Mechanism	11
2.7.2	Query, Key, Value (Q, K, V)	11
2.7.3	Scaled Dot-Product Attention	11
2.7.4	Multi-Head Attention	12
2.8	Transformer Architecture	12
2.8.1	Encoder	12
2.8.2	Decoder	13
2.8.3	Positional Encoding	13
2.8.4	Output Layer	13
2.8.5	Label Smoothing	14
2.8.6	Key Parameters (Original Paper)	14
3	End-to-End Translation Pipeline	14
4	Related Concepts You Should Also Know	16
5	Build Projects	17
6	Explain It Back	18

7 Interview Practice Questions & Answers	19
8 Self-Assessment Test	21

Topic Map

This lecture covers the foundational pipeline from raw text to the Transformer architecture. Below is the complete hierarchy of topics and sub-topics.

1. NLP Overview

- 1.1 Classification tasks (sentiment extraction, intent detection, language detection, topic modeling)
- 1.2 Multi-classification tasks (NER, POS tagging, dependency/constituency parsing)
- 1.3 Generation tasks (machine translation, question answering, summarization, code/text generation)
- 1.4 Evaluation metrics (accuracy, precision, recall, F1, BLEU, ROUGE, perplexity)

2. Tokenization

- 2.1 Word-level tokenization
- 2.2 Subword-level tokenization (BPE, WordPiece)
- 2.3 Character-level tokenization
- 2.4 Out-of-Vocabulary (OOV) problem

3. Token/Word Representation

- 3.1 One-Hot Encoding (OHE) and its limitations
- 3.2 Cosine similarity
- 3.3 Word2vec: CBOW and Skip-gram
- 3.4 Proxy tasks and learned embeddings
- 3.5 Embedding dimensions and vocabulary size

4. Sequence Models (Pre-Transformer)

- 4.1 Recurrent Neural Networks (RNNs) — architecture, hidden state, use cases
- 4.2 Long Short-Term Memory (LSTM) — cell state, gating
- 4.3 Vanishing/exploding gradient problem
- 4.4 Long-range dependency limitations

5. Attention Mechanism

- 5.1 Motivation: solving long-range dependencies
- 5.2 Bahdanau attention (2014) — direct links between encoder and decoder
- 5.3 Self-attention — all tokens attend to all tokens simultaneously

6. Transformer Architecture

- 6.1 Query, Key, Value (Q, K, V) formulation
- 6.2 Scaled dot-product attention: $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
- 6.3 Multi-head attention — parallel projections
- 6.4 Encoder: self-attention $\rightarrow$ FFN

6.5 Decoder: masked self-attention $\rightarrow$ cross-attention $\rightarrow$ FFN

6.6 Positional encoding (sinusoidal)

6.7 Label smoothing

6.8 End-to-end translation pipeline

Deep-Dive Notes

NLP Task Taxonomy

Definition: Natural Language Processing (NLP)

NLP is the field concerned with computational manipulation and understanding of human language (text and speech). At its core, NLP converts unstructured text into structured representations that models can process.

NLP tasks fall into three buckets based on the relationship between inputs and outputs:

Classification (Text $\rightarrow$ Single Label)

The model receives an input text and predicts a single categorical label.

Example: Sentiment Analysis

Input: "This teddy bear is SO CUTE!" $\rightarrow$ Output: Positive

Common datasets: IMDb reviews, Amazon product reviews, Twitter/X posts.

Other tasks in this bucket: intent detection (e.g., extracting “create alarm” from a user command), language detection, and topic modeling.

Multi-Classification (Text $\rightarrow$ Multiple Labels per Token)

The model receives text and produces a label for each token (or a subset of tokens).

Definition: Named Entity Recognition (NER)

Given an input sequence, NER identifies and classifies named entities into predefined categories such as PERSON, LOCATION, ORGANIZATION, TIME, etc. Each token receives a label.

Evaluation is done at the *token level* or per *entity type*, not at the sentence level. Other tasks: Part-of-Speech (POS) tagging, dependency parsing, constituency parsing.

Generation (Text $\rightarrow$ Variable-Length Text)

The model receives text and produces text of arbitrary length. This is the dominant paradigm today.

Tasks: machine translation, question answering (ChatGPT, Gemini), summarization, code generation, creative writing.

Evaluation Metrics

Classification Metrics

Precision, Recall, F1

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (\text{Of all predicted } +, \text{ how many correct?}) \quad (1)$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (\text{Of all actual } +, \text{ how many caught?}) \quad (2)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{Harmonic mean}) \quad (3)$$

Gotcha: Class Imbalance

Accuracy is misleading when classes are imbalanced. If 99% of samples are positive, a model that always predicts positive achieves 99% accuracy but 0% recall on the minority class. Always check precision, recall, and F1 per class.

Generation Metrics**BLEU (Bilingual Evaluation Understudy)**

Measures *precision-like* quality of generated text against reference translations. Computes n-gram overlap between candidate and reference(s). Higher is better. Originally designed for machine translation.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

A suite of metrics measuring *recall-like* overlap. ROUGE-N measures n-gram recall, ROUGE-L measures longest common subsequence. Higher is better. Commonly used for summarization.

Perplexity (PPL)

Quantifies how “surprised” a language model is by a sequence. Mathematically:

$$\text{PPL}(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \ln P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

Lower is better. A perplexity of k means the model is, on average, as uncertain as if it were choosing uniformly among k words at each step.

Gotcha: Reference-Based vs. Reference-Free

BLEU and ROUGE require reference texts (labels), which are expensive to produce. Modern LLM evaluation increasingly uses reference-free metrics (e.g., LLM-as-judge). Perplexity requires no reference but only measures model confidence, not output quality.

Tokenization

Definition: Tokenization

The process of decomposing raw text into discrete units called **tokens**. Tokens are the atomic inputs to any NLP model. The choice of tokenization strategy directly impacts vocabulary size, sequence length, and model performance.

Word-Level Tokenization

Split on whitespace/punctuation. Each word is a token.

- **Pro:** Simple, interpretable.
- **Con:** Morphologically related words (“bear” vs. “bears”, “run” vs. “running”) are treated as completely separate tokens. High OOV risk at inference time.

Subword-Level Tokenization

Decomposes words into meaningful subword units by learning common prefixes, suffixes, and roots from a training corpus.

Example: WordPiece Tokenization

reading → [read, ##ing]

teddy → [ted, ##dy]

The ## prefix indicates a continuation piece (not the start of a word).

- **Pro:** Leverages root morphology; significantly lower OOV risk than word-level.
- **Con:** Longer sequences (more tokens per sentence ⇒ more computation).

Common algorithms: BPE (Byte Pair Encoding), WordPiece (used in BERT), SentencePiece (used in T5, LLaMA).

Character-Level Tokenization

Each character is a token.

- **Pro:** Virtually zero OOV; robust to misspellings and casing errors.
- **Con:** Extremely long sequences ⇒ very slow training/inference. Individual character embeddings carry minimal semantic meaning.

Definition: Out-of-Vocabulary (OOV)

A token encountered at inference time that was not present in the training vocabulary. Typically mapped to a special [UNK] (unknown) token, which collapses all unseen words into a single representation — losing all semantic information.

Practical Insight: Vocabulary Size

- Single-language models: ~30K–50K tokens (e.g., GPT-2: 50,257)
- Multilingual + code models: ~100K–256K tokens (e.g., LLaMA 3: 128K; GPT-4o: 200K)
- Larger vocabularies reduce sequence length but increase embedding table size ($V \times d_{\text{model}}$).

Table 1: Tokenization Comparison

Property	Word	Subword	Character
OOV Risk	High	Low	Near zero
Sequence Length	Short	Medium	Very long
Morphology Awareness	None	High	N/A
Compute Cost	Low	Medium	High
Typical Use	Legacy	Modern standard	Niche

Token Representation and Word2vec

One-Hot Encoding (OHE)

Definition: One-Hot Encoding

Each token in a vocabulary of size V is represented as a binary vector $\mathbf{x} \in \{0, 1\}^V$ where exactly one element is 1. Token i has a 1 at position i and 0 everywhere else.

Fundamental Problem

All one-hot vectors are orthogonal to each other: $\mathbf{x}_i^\top \mathbf{x}_j = 0$ for $i \neq j$. This means *no similarity information* is captured. “Teddy bear” and “soft” are as distant as “teddy bear” and “quantum mechanics.”

Cosine Similarity

Definition: Cosine Similarity

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Measures the angle between two vectors, ignoring magnitude. Range: $[-1, 1]$ where $1 =$ identical direction, $0 =$ orthogonal (independent), $-1 =$ opposite.

This is the standard similarity metric for comparing embeddings. The dot product alone is also used but is scale-sensitive.

Word2vec (Mikolov et al., 2013)

Definition: Word2vec

A shallow neural network trained on a **proxy task** (predicting words from context or vice versa) over large text corpora. The learned hidden-layer weights serve as dense, low-dimensional word embeddings of dimension $d \ll V$.

Definition: Proxy Task (Pretext Task)

A task whose primary purpose is not its own output but rather the useful representations learned during training. In Word2vec, predicting neighboring words is the proxy task; the real goal is learning embeddings.

Two training objectives:

1. **Continuous Bag of Words (CBOW):** Predict the target word from its surrounding context words.

Context: $[w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}] \rightarrow \text{Predict: } w_t$

2. **Skip-gram:** Predict the surrounding context words from the target word.

Target: $w_t \rightarrow \text{Predict: } [w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}]$

Architecture Walkthrough

Consider a vocabulary of size $V = 6$ and hidden dimension $d = 2$.

Step 1: Input word “A” as OHE vector $[1, 0, 0, 0, 0, 0]$.

Step 2: Multiply by weight matrix $W_1 \in \mathbb{R}^{V \times d}$ to get hidden state $\mathbf{h} = [0.2, 0.9]$.

Step 3: Multiply by $W_2 \in \mathbb{R}^{d \times V}$, apply softmax $\rightarrow$ probability distribution over vocabulary: $[0.2, 0.4, 0.1, 0.1, 0.1, 0.1]$.

Step 4: Compare with true next word (“cute” = $[0, 1, 0, 0, 0, 0]$), compute cross-entropy loss, backpropagate.

Result: After training, the hidden-layer vector $\mathbf{h}$ for each word *is* that word’s embedding. The rows of W_1 are the learned embeddings.

Key properties of Word2vec embeddings:

- Capture semantic relationships: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$
- Similar words cluster together in embedding space
- Typical embedding dimensions: $d = 100\text{--}300$ (Word2vec era), $d = 768\text{--}4096$ (modern transformers)

Limitations of Word2vec

- **Static embeddings:** Each word gets one fixed vector regardless of context. “Bank” (river) and “bank” (financial) have the same embedding.
- **No word order:** Bag-of-words assumption means “dog bites man” and “man bites dog” produce identical representations.
- Training convergence is tracked by monitoring the proxy-task loss over epochs; stop when it plateaus.

Recurrent Neural Networks (RNNs)

Definition: Recurrent Neural Network (RNN)

A class of neural networks where connections form a directed cycle over a temporal sequence. At each time step t , the network takes the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, producing a new hidden state:

$$\mathbf{h}_t = f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

where f is typically tanh or ReLU. First introduced in the 1980s.

The hidden state $\mathbf{h}_t$ encapsulates the “meaning of the sequence processed so far.” This is also called the **activation vector** or **context vector**.

RNN for Different NLP Tasks

- **Classification:** Use the final hidden state $\mathbf{h}_T$ and project to label space (e.g., positive/negative).
- **Multi-classification:** Use each hidden state $\mathbf{h}_t$ to predict a label for token t .
- **Generation (seq2seq):** Encode the full input into $\mathbf{h}_T$ (context vector), then decode token by token using a separate decoder RNN.

LSTM (Long Short-Term Memory)**Definition: LSTM (Hochreiter & Schmidhuber, 1997)**

An extension of RNNs that introduces a **cell state** $\mathbf{c}_t$ in addition to the hidden state $\mathbf{h}_t$. The cell state acts as a “memory highway” regulated by three gates:

- **Forget gate** f_t : decides what to discard from cell state
- **Input gate** i_t : decides what new information to store
- **Output gate** o_t : decides what to output from cell state

This allows the network to selectively remember or forget information over long sequences.

The Vanishing Gradient Problem**Definition: Vanishing Gradient**

During backpropagation through time (BPTT), gradients are computed as a product of Jacobians across time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} \propto \prod_{t=1}^T \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$$

If the spectral radius of each Jacobian is < 1 , the product vanishes exponentially. If > 1 , it explodes. In either case, the model cannot effectively learn long-range dependencies.

Why RNNs Fell Out of Favor

1. **Long-range dependencies:** Even LSTMs struggle beyond ~ 500 tokens.
2. **Sequential computation:** To compute $\mathbf{h}_T$, you must compute $\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_{T-1}$ first. No parallelism $\Rightarrow$ very slow training on GPUs.
3. **Information bottleneck:** The entire input sequence is compressed into a single fixed-size vector $\mathbf{h}_T$.

Attention Mechanism**Definition: Attention (Bahdanau et al., 2014)**

A mechanism that creates **direct connections** between the decoder at each time step and all encoder hidden states. Instead of relying solely on the final encoder hidden state, the decoder can “attend to” (peek at) relevant parts of the input at each decoding step.

Intuition: Translation

When translating “A cute teddy bear is reading” to French, while generating the word “mignon” (cute), the model should attend most strongly to the input word “cute.” Attention provides this direct link, bypassing the bottleneck of compressing everything into one vector.

This was the direct precursor to the Transformer. The key insight: *direct connections solve long-range dependencies better than sequential processing.*

Self-Attention and the Transformer

Self-Attention Mechanism

Definition: Self-Attention

Unlike Bahdanau attention (decoder attends to encoder), self-attention allows **every token in a sequence to attend to every other token in the same sequence**. This produces context-aware representations where the embedding of “bank” differs depending on whether “river” or “robbery” appears nearby.

Introduced as the core mechanism in “Attention Is All You Need” (Vaswani et al., 2017).

Query, Key, Value (Q, K, V)

Definition: Query, Key, Value

Each input embedding $\mathbf{x}_i$ is projected into three different spaces:

$$\mathbf{q}_i = \mathbf{W}_Q \mathbf{x}_i \quad (\text{Query: “what am I looking for?”}) \quad (4)$$

$$\mathbf{k}_i = \mathbf{W}_K \mathbf{x}_i \quad (\text{Key: “what do I contain?”}) \quad (5)$$

$$\mathbf{v}_i = \mathbf{W}_V \mathbf{x}_i \quad (\text{Value: “what information do I provide?”}) \quad (6)$$

$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ are **learned** projection matrices.

Intuition: Dictionary Lookup Analogy

Think of self-attention as a soft dictionary lookup. You have a query (the word you want to contextualize). You compare that query against all keys (all other words). The most similar keys get higher weights. You then take a weighted sum of the corresponding values to produce the contextualized representation.

Scaled Dot-Product Attention

Formula: Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where:

- $Q \in \mathbb{R}^{n \times d_k}$ — matrix of all queries (one row per token)
- $K \in \mathbb{R}^{n \times d_k}$ — matrix of all keys
- $V \in \mathbb{R}^{n \times d_v}$ — matrix of all values
- $\sqrt{d_k}$ — scaling factor to prevent dot products from growing too large

Step-by-Step Computation

Step 1: Compute $QK^\top \in \mathbb{R}^{n \times n}$. Entry (i, j) = dot product of query i with key j = how much token i should attend to token j .

Step 2: Divide by $\sqrt{d_k}$. As d_k grows, dot products grow in magnitude, pushing softmax into regions of tiny gradients. Scaling prevents this.

Step 3: Apply softmax row-wise $\rightarrow$ each row becomes a probability distribution over all tokens (attention weights).

Step 4: Multiply by $V \rightarrow$ each output row is a weighted sum of all value vectors, weighted by the attention scores from Step 3.

Why $\sqrt{d_k}$? (Scaling Factor)

If q and k are random vectors with independent components of mean 0 and variance 1, their dot product has mean 0 and variance d_k . Dividing by $\sqrt{d_k}$ normalizes the variance back to 1, keeping the softmax in a numerically stable regime. Without this, for large d_k , softmax saturates and gradients vanish.

Multi-Head Attention

Definition: Multi-Head Attention (MHA)

Instead of performing a single attention computation, run h attention computations in parallel, each with its own learned projection matrices ($\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)}, \mathbf{W}_V^{(i)}$):

$$\text{head}_i = \text{Attention}(QW_Q^{(i)}, KW_K^{(i)}, VW_V^{(i)})$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

W_O projects the concatenated heads back to d_{model} dimensions.

- Each head can learn different “types” of attention (e.g., one head for syntactic relationships, another for semantic, another for positional).
- Analogous to multiple filters in a CNN layer.
- No explicit constraint forces heads to be different; gradient descent naturally diversifies them because identical heads would be wasteful degrees of freedom.

Transformer Architecture

The full Transformer (Vaswani et al., 2017) is an encoder–decoder architecture designed for sequence-to-sequence tasks (originally machine translation).

Encoder

The encoder processes the input sequence and produces context-aware representations.

Encoder Block (repeated N times)

1. **Multi-Head Self-Attention:** Every input token attends to every other input token. Produces contextualized representations.
2. **Add & Norm:** Residual connection + Layer Normalization.
3. **Feed-Forward Network (FFN):** Two linear transformations with a ReLU in between:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + b_1)W_2 + b_2$$

The hidden dimension d_{FF} is typically $4 \times d_{\text{model}}$ (e.g., $d_{\text{model}} = 512 \Rightarrow d_{\text{FF}} = 2048$).

4. **Add & Norm:** Another residual connection + Layer Normalization.

Why is $d_{\text{FF}} > d_{\text{model}}$?

In Word2vec, the hidden layer was *smaller* than the vocabulary (compression). In the Transformer FFN, the hidden layer is *larger* than d_{model} (expansion). The rationale: the FFN needs enough capacity to learn complex, non-linear transformations of the attention outputs. The bottleneck-expansion-bottleneck pattern ($d_{\text{model}} \rightarrow d_{\text{FF}} \rightarrow d_{\text{model}}$) gives the network expressive power while keeping the residual stream at a manageable dimension.

Decoder

The decoder generates the output sequence one token at a time.

Decoder Block (repeated N times)

1. **Masked Multi-Head Self-Attention:** Each token attends only to previously generated tokens (and itself). Future tokens are masked to $-\infty$ before softmax, ensuring autoregressive generation.
2. **Add & Norm**
3. **Multi-Head Cross-Attention:**
 - **Queries** come from the decoder (previous layer's output)
 - **Keys and Values** come from the encoder's final output

This lets the decoder “look at” the full input while deciding what to generate next.
4. **Add & Norm**
5. **FFN + Add & Norm**

Cross-Attention Intuition

When translating to French and generating the word “mignon”: the decoder's query is derived from the tokens decoded so far (“Un ours en peluche”). The keys and values come from the encoder's representation of “A cute teddy bear is reading.” The model learns to attend strongly to “cute” when generating “mignon.”

Positional Encoding**Definition: Positional Encoding**

Since self-attention has no inherent notion of token order (it processes all tokens simultaneously via matrix multiplication), position information must be explicitly injected. The original Transformer uses sinusoidal functions:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (7)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (8)$$

These are added element-wise to the token embeddings.

Properties: deterministic (not learned), can generalize to unseen sequence lengths, and the relative position between any two positions can be represented as a linear function of the encodings.

Output Layer

After N decoder blocks, the output goes through:

1. A linear projection to vocabulary size V

2. Softmax to get probability distribution over the vocabulary
3. The highest-probability token is selected (or sampled)

Generation stops when the model outputs the [EOS] (End Of Sequence) special token.

Label Smoothing

Definition: Label Smoothing

Instead of training against hard one-hot targets $[0, 0, 1, 0, \dots, 0]$, use soft targets:

$$y_i = \begin{cases} 1 - \epsilon & \text{if } i = \text{correct class} \\ \frac{\epsilon}{V-1} & \text{otherwise} \end{cases}$$

where ϵ is typically small (e.g., 0.1). This acknowledges that in language, multiple valid continuations exist.

- Prevents the model from becoming overconfident.
- Acts as a regularizer, reducing overfitting.
- Empirically improves BLEU scores on translation tasks.

Key Parameters (Original Paper)

Table 2: Transformer Base Configuration (Vaswani et al., 2017)

Parameter	Value
N (encoder/decoder layers)	6
d_{model} (embedding dimension)	512
d_{FF} (FFN hidden dimension)	2048
h (number of attention heads)	8
$d_k = d_v = d_{\text{model}}/h$	64
Label smoothing ϵ	0.1

End-to-End Translation Pipeline

Tying it all together for translating “A cute teddy bear is reading” (English $\rightarrow$ French):

1. **Tokenize:** Split into tokens, add special tokens: [BOS] A cute teddy bear is reading . [EOS]
2. **Embed:** Look up learned embeddings for each token (d_{model} -dimensional vectors).
3. **Add Positional Encoding:** Element-wise addition of sinusoidal position vectors.
4. **Encode:** Pass through N encoder blocks. Each block: self-attention $\rightarrow$ FFN. Output: context-aware embeddings for every input token.
5. **Decode (autoregressive):**
 - (a) Start with [BOS] token.
 - (b) Masked self-attention over decoded tokens so far.
 - (c) Cross-attention: queries from decoder, keys/values from encoder output.

- (d) FFN, then linear + softmax $\rightarrow$ predict next token: “Un.”
 - (e) Feed “Un” back into decoder. Repeat: predict “ours,” “en,” “peluche,” “mignon,” “lit,” [EOS].
6. **Stop:** Generation halts upon predicting [EOS].

Related Concepts You Should Also Know

These concepts were touched on or assumed by the lecture but not fully explained. Interviewers love probing these gaps.

1. **Byte Pair Encoding (BPE):** The most common subword tokenization algorithm. Iteratively merges the most frequent pair of adjacent tokens until reaching the desired vocabulary size. Used in GPT models.
2. **Residual Connections (Skip Connections):** The “Add” in “Add & Norm.” output = $\text{sublayer}(\mathbf{x}) + \mathbf{x}$. Allows gradients to flow directly through the network, mitigating vanishing gradients in deep stacks. Critical for training Transformers with $N \geq 6$ layers.
3. **Layer Normalization:** Normalizes activations across the feature dimension (not the batch dimension like BatchNorm). Stabilizes training by reducing internal covariate shift. Applied after each sublayer in the Transformer.
4. **Softmax Temperature:** Scaling logits by a temperature τ before softmax: $\text{softmax}(z_i/\tau)$. High $\tau \rightarrow$ more uniform (exploratory); low $\tau \rightarrow$ more peaked (greedy). Used during inference/generation.
5. **Cross-Entropy Loss:** The standard loss function for next-token prediction: $\mathcal{L} = -\sum_i y_i \log(\hat{y}_i)$. With label smoothing, y_i is the smoothed distribution.
6. **Teacher Forcing:** During training, the decoder receives the ground-truth previous token (not its own prediction) as input. Speeds convergence but creates train–inference mismatch.
7. **Beam Search:** A decoding strategy that explores multiple candidate sequences simultaneously (beam width B). More diverse than greedy decoding but more expensive.
8. **GloVe (Global Vectors):** Another pre-transformer embedding method (Pennington et al., 2014). Uses co-occurrence matrix factorization rather than neural networks. Produces static embeddings like Word2vec.
9. **GRU (Gated Recurrent Unit):** A simplified variant of LSTM with two gates instead of three. Often comparable performance with fewer parameters.
10. **Encoder-Only vs. Decoder-Only Architectures:** The full encoder–decoder Transformer is for seq2seq tasks. BERT uses encoder-only (for classification, NER). GPT uses decoder-only (for generation). This is covered in later lectures.

Build Projects

These projects force you to apply the concepts from this lecture. Ranked by portfolio signal-to-effort ratio.

1. **Tokenizer Comparison Benchmark** (*2–4 hours, high signal*)

Implement word-level, BPE, and character-level tokenizers from scratch (no libraries). Benchmark them on a multilingual corpus: measure vocabulary size, average sequence length, OOV rate, and tokenization speed. Visualize trade-offs. Write it up as a Jupyter notebook with analysis. *Demonstrates:* deep understanding of the tokenization-compute trade-off.

2. **Attention Visualizer** (*4–6 hours, high signal*)

Build an interactive tool (React or Streamlit) that takes a sentence, runs it through a pre-trained Transformer (e.g., BERT via HuggingFace), and visualizes the attention weights as heatmaps for each head and layer. Let users click on a token and see what it attends to. *Demonstrates:* understanding of multi-head attention internals.

3. **Mini Transformer from Scratch** (*8–12 hours, highest signal*)

Implement a small encoder–decoder Transformer in PyTorch (no HuggingFace). Train it on a toy translation task (e.g., reversing sequences, or translating numbers to words). Include: positional encoding, multi-head attention, masking, label smoothing. This is the gold-standard portfolio piece for “I actually understand Transformers.” *Tip:* Andrej Karpathy’s “minGPT” and “nanoGPT” are great references for decoder-only; extend to encoder–decoder.

Explain It Back

Write or speak your answers to these prompts *without looking at notes*. If you cannot explain it clearly, you do not understand it yet.

1. Explain to a junior engineer why subword tokenization is preferred over word-level tokenization. Use the example of “playing,” “plays,” and “played.”
2. A product manager asks you: “Why can’t we just use RNNs? They process sequences too.” Give a 2-minute explanation of why Transformers replaced them, covering both quality (long-range dependencies) and efficiency (parallelism).
3. Explain the Q, K, V mechanism to someone who has never seen it. Use a real-world analogy (the lecture uses dictionary lookup; try a different one — e.g., a search engine, a library catalog, or a party where you’re looking for the most interesting conversation).
4. Why does the Transformer need positional encoding? What would go wrong without it? Give a concrete example of two sentences that would be indistinguishable.

Interview Practice Questions & Answers

Questions are tiered: **L1** = screening/phone, **L2** = mid-round, **L3** = deep-dive/follow-up.

L1 — What is the difference between self-attention and cross-attention?

Answer: In self-attention, queries, keys, and values all come from the same sequence. Each token attends to every other token in that sequence, producing context-aware representations. In cross-attention, queries come from one sequence (e.g., the decoder) while keys and values come from a different sequence (e.g., the encoder output). Cross-attention is how the decoder “looks at” the input when deciding what to generate next.

L1 — Why do we scale dot products by $\sqrt{d_k}$?

Answer: As the dimension d_k increases, the dot product of two random vectors grows in variance (proportional to d_k). Without scaling, the softmax receives very large inputs, pushing it into saturation regions where gradients are near zero. Dividing by $\sqrt{d_k}$ normalizes the variance to approximately 1, keeping softmax numerically stable and gradients healthy.

L2 — What is the computational complexity of self-attention, and why is it a concern?

Answer: The $QK^\top$ matrix multiplication produces an $n \times n$ matrix where n is sequence length. This gives $O(n^2 \cdot d)$ time complexity and $O(n^2)$ memory for the attention matrix. For long sequences (e.g., 100K tokens), this becomes prohibitive. This is why methods like FlashAttention, sparse attention, and linear attention have been developed.

L2 — Explain why multi-head attention is used instead of single-head attention with the same total dimension.

Answer: Multi-head attention runs h parallel attention operations, each with dimension $d_k = d_{\text{model}}/h$. Each head can learn different attention patterns (syntactic, semantic, positional). A single head with dimension d_{model} would compute one attention pattern. With multiple heads, the model captures richer, more diverse relationships. The total computation is roughly the same (since each head operates on d_k instead of d_{model}), but representational capacity increases.

L2 — What is the masking in the decoder’s self-attention, and why is it necessary?

Answer: The decoder generates tokens autoregressively (left to right). During training, all target tokens are available simultaneously for efficiency (teacher forcing), but each position should only attend to positions $\leq t$ (not future tokens). The mask sets future positions to $-\infty$ before softmax, making their attention weights zero. Without masking, the model would “cheat” by looking at tokens it has not yet generated, breaking the autoregressive property.

L3 — What would happen if you removed positional encoding entirely? How would performance degrade?

Answer: Without positional encoding, the Transformer treats the input as a *bag of tokens*. It cannot distinguish “dog bites man” from “man bites dog.” Self-attention alone is permutation-equivariant: permuting the input tokens permutes the output identically. Performance degrades severely on any task requiring word order understanding (nearly all NLP

tasks). The model would still capture which tokens co-occur (unigram-like statistics) but lose all sequential/structural information.

L3 — Explain the information flow from a single input token through one full encoder layer.

Answer: Start with token embedding $\mathbf{x}$ + positional encoding. (1) Project $\mathbf{x}$ into Q, K, V via learned matrices. (2) Compute attention: $\text{softmax}(QK^\top/\sqrt{d_k})V$, yielding a context-weighted representation. (3) Add residual: $\mathbf{x}' = \mathbf{x} + \text{Attention}(\mathbf{x})$. (4) Layer norm: $\mathbf{x}'' = \text{LN}(\mathbf{x}')$. (5) FFN: expand to d_{FF} , ReLU, project back to d_{model} . (6) Add residual: $\mathbf{x}''' = \mathbf{x}'' + \text{FFN}(\mathbf{x}'')$. (7) Layer norm again. Output is a context-enriched representation of the original token.

L2 — How does label smoothing act as regularization?

Answer: Without label smoothing, the model is trained to push the probability of the correct token to 1.0 and all others to 0.0. This forces logits toward infinity, creating overconfident predictions that generalize poorly. Label smoothing sets the target to $1 - \epsilon$ for the correct token and distributes ϵ across all other tokens. This keeps logits bounded, produces softer probability distributions, and acts as a regularizer that discourages the model from being too certain — which aligns with the reality that language has multiple valid continuations.

L1 — What is a proxy task? Give two examples.

Answer: A proxy task (or pretext task) is a training objective whose primary goal is not the task itself but the useful representations learned during training. Examples: (1) Word2vec's CBOW/Skip-gram — predicting surrounding words teaches the model meaningful word embeddings. (2) Masked Language Modeling in BERT — predicting masked tokens forces the model to learn deep bidirectional context representations.

L3 — Compare the information bottleneck in seq2seq RNNs vs. Transformers.

Answer: In seq2seq RNNs, the entire input sequence is compressed into a single fixed-size vector $\mathbf{h}_T$ (the encoder's final hidden state). All information must pass through this bottleneck, and information from early tokens is often lost. In Transformers, the encoder produces n separate context-aware vectors (one per input token), and the decoder's cross-attention can directly access any of them at each step. There is no single bottleneck vector. This is fundamentally why Transformers handle long sequences and complex dependencies better.

Self-Assessment Test

Instructions: Answer without looking at notes. Score yourself honestly.

Scoring: 16–20 correct = solid mastery; 12–15 = good, revisit weak spots; <12 = re-study before moving on.

Multiple Choice (1 point each)

Q1. Which tokenization method has the highest risk of OOV tokens?

- (a) Character-level (b) Subword-level (c) Word-level (d) BPE

Q2. In Word2vec's Skip-gram, the model predicts:

- (a) The target word from context (b) Context words from the target (c) The next sentence
(d) Named entities

Q3. What is the dimensionality of the attention score matrix $QK^\top$?

- (a) $n \times d_k$ (b) $d_k \times d_k$ (c) $n \times n$ (d) $n \times d_v$

Q4. In the decoder's cross-attention, the queries come from:

- (a) The encoder (b) The decoder (c) Both equally (d) The input embeddings directly

Q5. Perplexity of a perfect model that always assigns probability 1 to the correct token is:

- (a) 0 (b) 1 (c) ∞ (d) Undefined

Q6. Which problem does the LSTM's cell state primarily address?

- (a) Computational speed (b) Vanishing gradients / long-range memory (c) Vocabulary size
(d) Tokenization

Q7. Label smoothing replaces the target $[0, 0, 1, 0]$ with approximately:

- (a) $[0.25, 0.25, 0.25, 0.25]$ (b) $[0.03, 0.03, 0.9, 0.03]$ (c) $[0, 0, 0.9, 0.1]$ (d) $[0.1, 0.1, 0.8, 0]$

Q8. Why do Transformers need positional encoding but RNNs do not?

- (a) RNNs have larger vocabulary (b) RNNs process tokens sequentially, inherently encoding order
(c) Transformers cannot learn (d) Positional encoding is optional in Transformers

Short Answer (1.5 points each)

Q9. In one sentence, explain why accuracy alone is insufficient for evaluating a sentiment classifier on an imbalanced dataset.

Q10. Write the scaled dot-product attention formula and label each component.

Q11. What is the fundamental difference between Bahdanau attention (2014) and self-attention (2017)?

Q12. Why is the FFN hidden dimension in the Transformer ($d_{\text{FF}} = 2048$) larger than $d_{\text{model}} = 512$?

Q13. Explain in two sentences what happens when you remove the mask from the decoder's self-attention during training.

Q14. Name two reasons why RNNs are slower to train than Transformers.

Scenario Question (2 points)

Q15. You are building a multilingual chatbot that must handle English, Hindi, Chinese, and code. The current word-level tokenizer has a vocabulary of 40K and users report frequent

[UNK] tokens for Hindi and Chinese inputs. Propose a solution, justify your tokenization choice, estimate the vocabulary size you would target, and explain one trade-off.

Answer Key

Q1. (c) Word-level

Q2. (b) Context words from the target

Q3. (c) $n \times n$

Q4. (b) The decoder

Q5. (b) $1 - \exp(-\frac{1}{N} \sum \ln(1)) = \exp(0) = 1$

Q6. (b) Vanishing gradients / long-range memory

Q7. (b) $[0.03, 0.03, 0.9, 0.03]$ — for $\epsilon = 0.1$, correct class gets 0.9, others get $0.1/3 \approx 0.03$

Q8. (b) RNNs process tokens sequentially, inherently encoding order

Q9. A model predicting only the majority class achieves high accuracy but zero recall on the minority class, making it useless for detecting rare-class instances.

Q10. $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$ where Q = queries, K = keys, V = values, d_k = key dimension, $QK^\top$ = attention scores, softmax = normalization to probability distribution.

Q11. Bahdanau attention connects a decoder to all encoder states (cross-sequence). Self-attention connects every token to every other token *within the same sequence*, enabling context-aware representations without an RNN.

Q12. The expanded hidden dimension gives the FFN enough capacity to learn complex non-linear transformations. It acts as a bottleneck-expansion-bottleneck pattern, providing additional degrees of freedom beyond what attention alone captures.

Q13. Without the mask, the decoder can see future tokens during training, meaning it can trivially copy the next token instead of learning to predict it. This breaks the autoregressive property and the model fails to generate coherently at inference time when future tokens are unavailable.

Q14. (1) RNNs process tokens sequentially — $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$ — so computation cannot be parallelized across time steps. (2) Backpropagation through time (BPTT) requires unrolling the full sequence, consuming memory proportional to sequence length and compounding numerical issues.

Q15. Switch to a subword tokenizer (SentencePiece or BPE) trained on a multilingual corpus covering all four languages plus code. Target vocabulary $\sim 128\text{K}$ – 200K tokens. This dramatically reduces OOV since subword decomposition handles unseen words via known subword pieces. Trade-off: larger vocabulary increases the embedding table size ($V \times d_{\text{model}}$) and softmax computation at the output layer, requiring more memory and slightly slower inference per token.

End of Study Guide — Lecture 1

Source: Stanford CME 295, Fall 2025. Afshine & Shervine Amidi.

Document prepared for self-study and interview preparation.